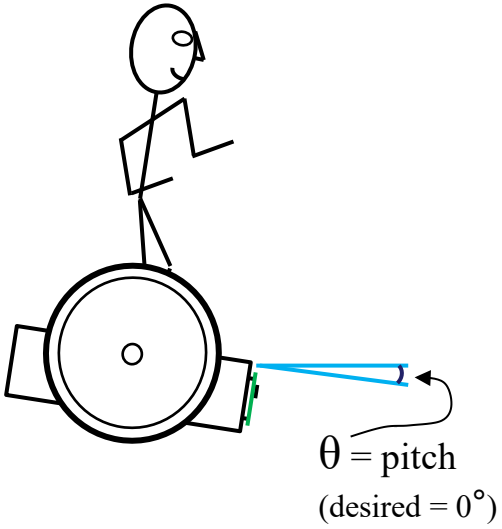
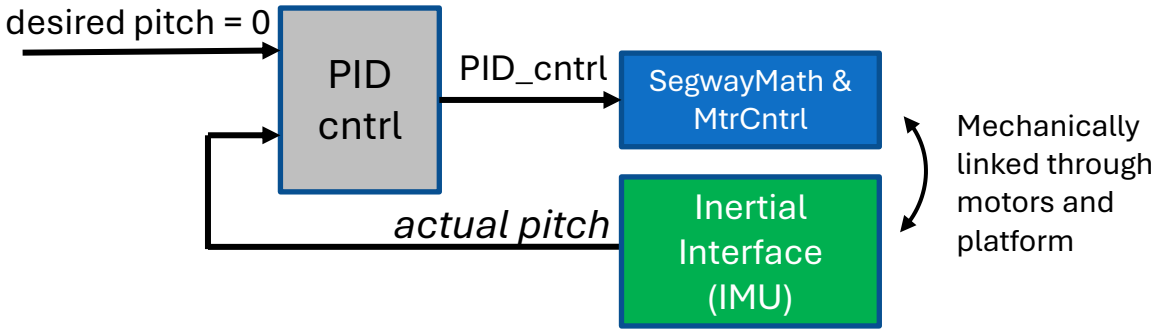
Exercise 06: PID Math

What is PID control?

- Common control scheme





For a segway:

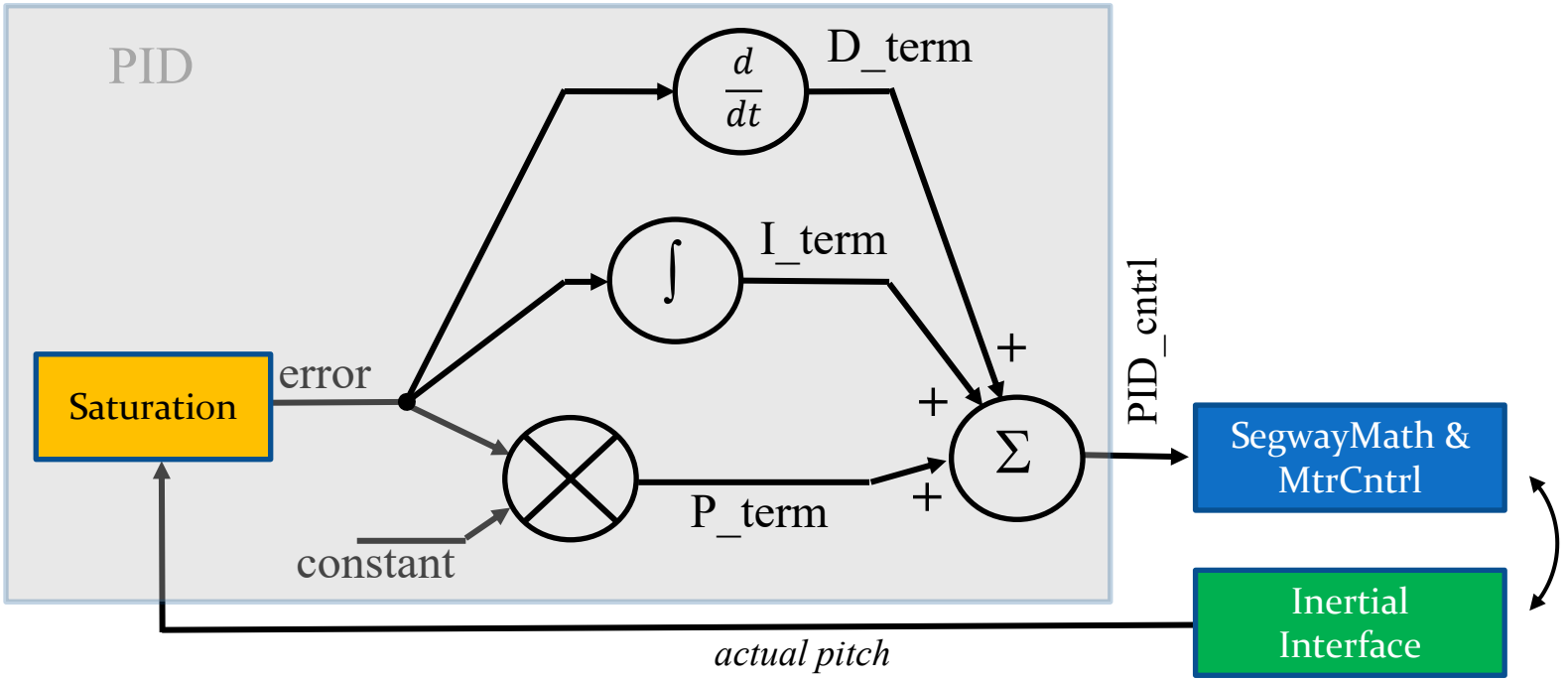
- We want to drive the left & right motors to balance the platform
- Our actual control knob (duty cycle of the PWM) controls the motor speed/torque

- The pitch of the platform is measured via an inertial sensor
- The desired pitch (zero) can be achieved by driving the motors forward/reverse the correct amount
- Determining the duty cycle to drive the motors frwrd/rev is the job of the PID block

Exercise 06: PID Math

What is PID control?

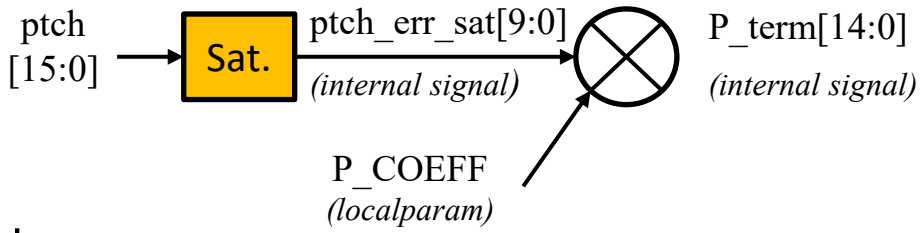
- PID → **P**roportional, **I**ntegral, **D**erivative



- We will get into the details of designing the PID block in a later exercise
- For this exercise, we will just "practice" some of the math that needs to occur inside the PID unit.

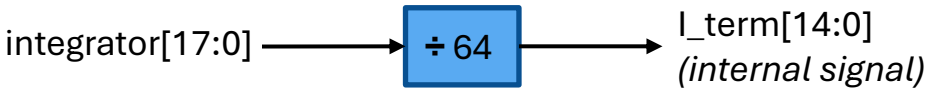
Exercise 06: PID Math

- Forming saturated 10-bit error term & P term
  - PID controllers work on an error term (difference between measured control variable and the desired). Since our control variable (pitch of the platform) is always **zero**, there is no need to subtract the desired from the actual to form an error term.
  - However, we do want to saturate the incoming signed 16-bit **ptch** to a signed 10-bit **ptch_err_sat** term
  - (P=proportional) **P_term** is simply formed by multiplying the error term by a coefficient.



P_COEFF = signed 5'h09
(subject to change)

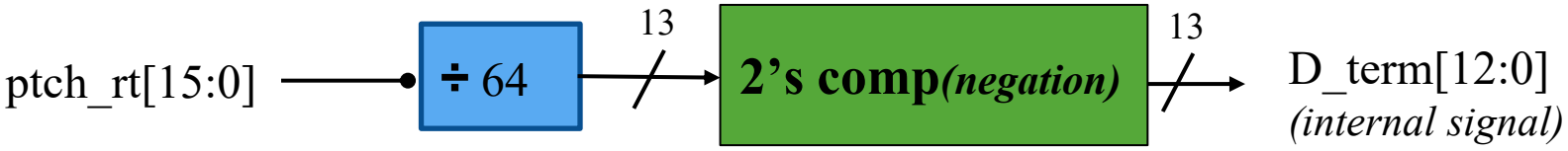
- I_term
  - Integration is summing over time. We will create the integrator accumulator in a later exercise, so we will assume we already have an 18-bit signed signal called **integrator[17:0]**
  - **I_term** will be a 15-bit quantity and is simply **integrator**/64
    (although 18-bit divided by 64 should be able to fit in 12 bits, we will modify some math later to speed simulation up. For this reason we will keep I_term at 15 bits)



What is the best way to divide by 64?

Exercise 06: PID Math

- D_term
  - Differentiation is not too difficult, but in our case, the inertial sensor actually produces angular rate of the platform.
  - D_term is simply proportional to the angular rate (**ptch_rt**) coming out of the inertial sensor (a MEMs gyro): **D_term = -(ptch_rt/64)**

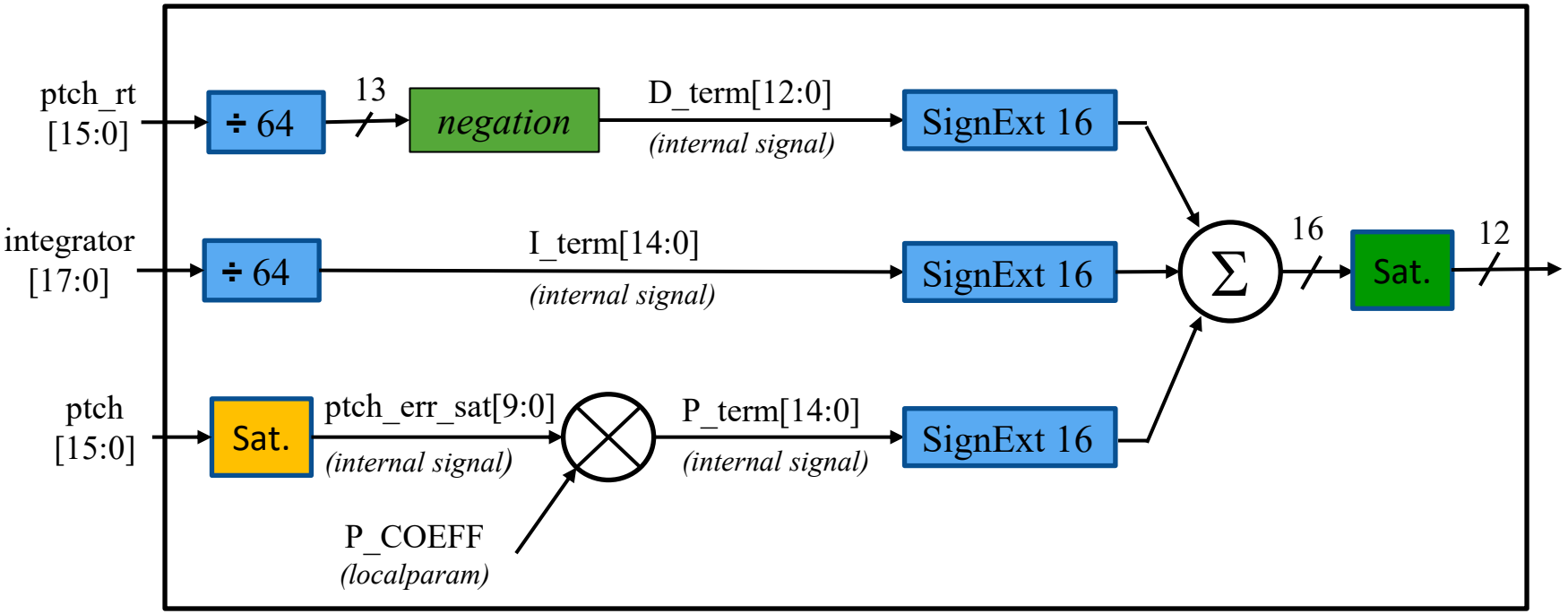


  - **D_term** is a 13-bit result so you need to sign extend **ptch_rt**
    (again, divide by 64 makes it 9 bits, but we are doing other things to it later, so keep it at 13)

- PID all together
  - **PID_cntrl** is simply the sum of **P_term**, **I_term**, **D_term** saturated to 12 bits

Exercise 06: PID Math

- Overall, we implement the following:

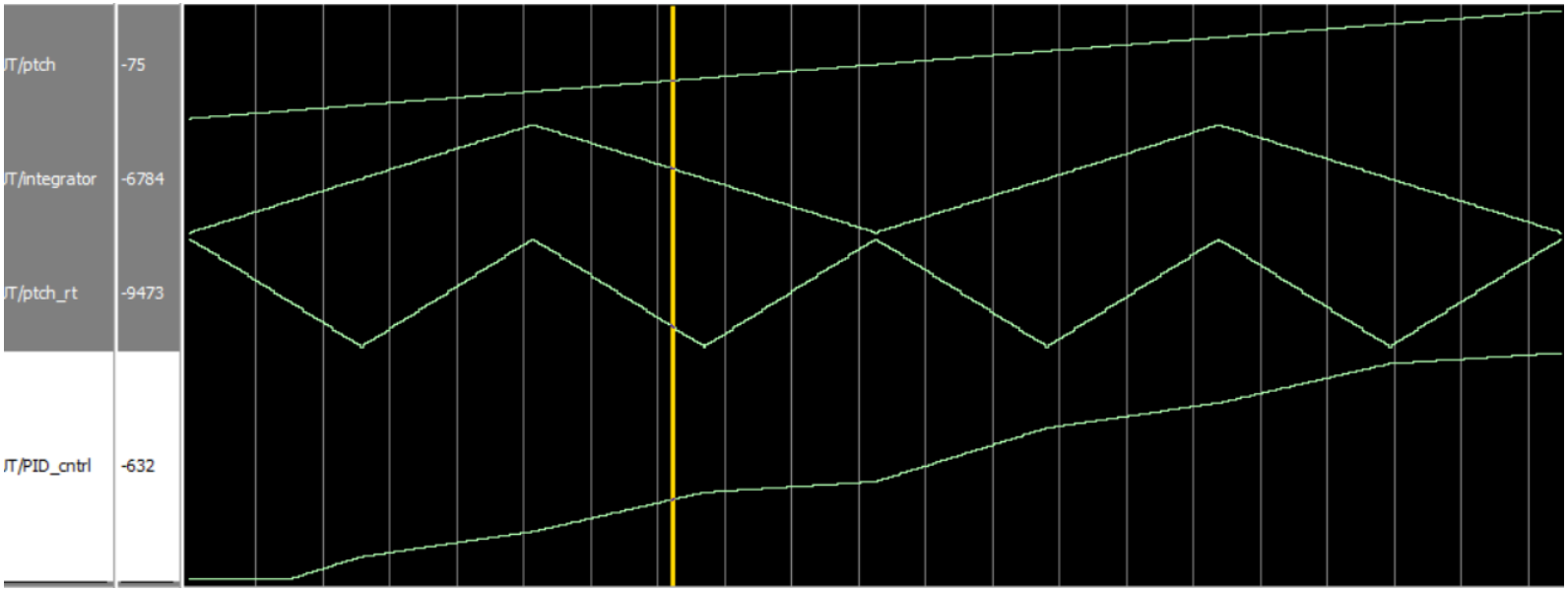


Signal:	Dir:	Description:
ptch[15:0]	in	Signed 16-bit pitch signal from **inertial_interface**.
ptch_rt[15:0]	in	Signed 16-bit pitch rate from **inertial_interface**. Used for D_term.
integrator[17:0]	in	18-bit integrator accumulation register
PID_cntrl[11:0]	out	12-bit signed result of PID control

Exercise 06: PID Math

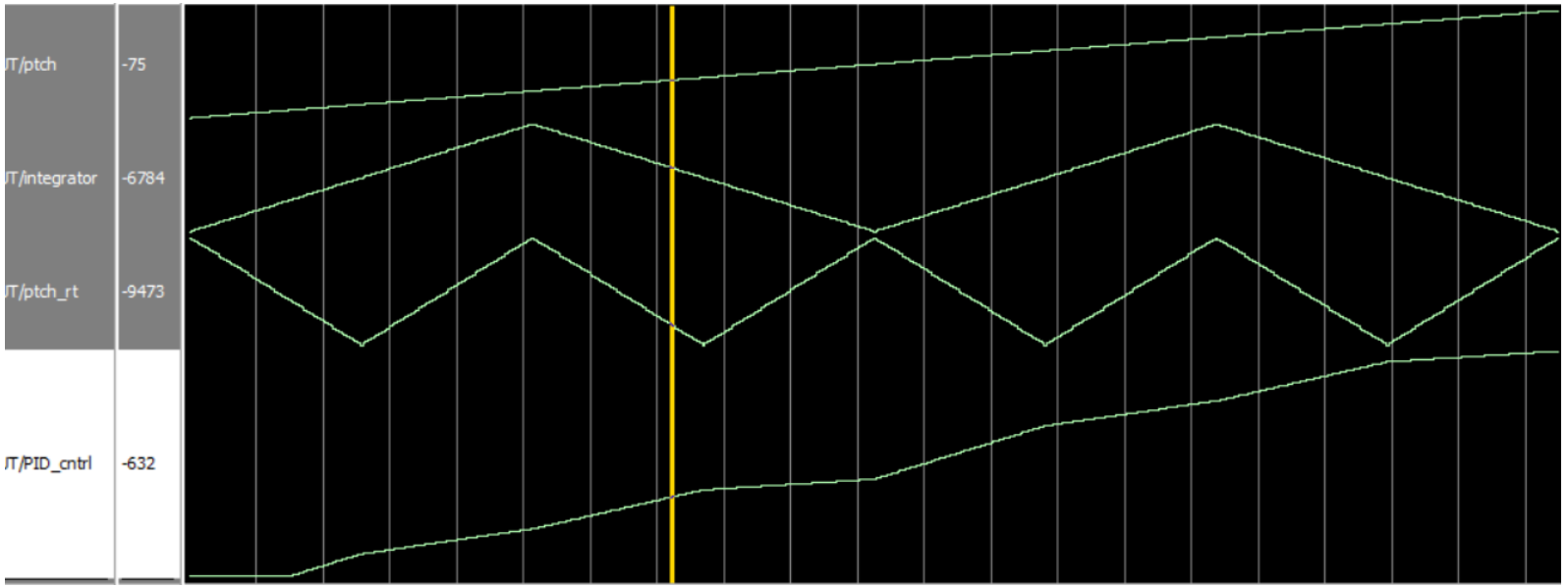
Testing – analog waveforms

This is a simulation where **ptch** starts at 16'hFF00 (-256), ramps to 16'h0FF (+255). Simultaneously **integrator** is ramping up/down four times from 18'h3C000 to 18'h03FFF and then back down to 18'h3C000. While all that is occurring **ptch_rt** is ramping down/up eight times with extents of its ramp being 16'h0FFF to 16'hF000.



This is not a comprehensive self-checking test, but if you can duplicate the subtle shape and rough magnitudes of **PID_cntrl** you are probably in good shape. We will revisit validating PID later in the course. **NOTE**: The format of **PID_cntrl** is not shown as *"Analog (automatic)"* but rather as *"Analog (custom)"* with 150px high.

Exercise 06: PID Math



There are several ways you can accomplish generating this stimulus. One way is the following:

- Break it into **8 repeat loops**, each with 64 iterations (so 512 in all)
- **ptch** starts at -256, and during these iterations ptch is always increasing by 1
- **integrator** is increasing by 18'h00080 in the first 2 loops, decreasing in the next 2 loops, etc.
- **ptch_rt** is decreasing by 16'h0100 in the first loop, decreasing in the next loop, etc.

You are free to use other methods if you'd like.

Submit what you have (**PID_Math.sv** & **PID_Math_tb.sv**) by the end of class. You need to finish this for HW2.